Scientific African 26 (2024) e02374

Contents lists available at ScienceDirect

Scientific African

journal homepage: www.elsevier.com/locate/sciaf

Stock price prediction using combined GARCH-AI models

John Kamwele Mutinda, Amos Kipkorir Langat *

African Master's In Machine Intelligence (AMMI), Senegal
Pan African University Institute for Basic Sciences, Technology and Innovation-JKUAT, Department of Mathematics, Nairobi, Kenya

ARTICLE INFO

Editor name: B. Gyampoh

Keywords:
LSTM
GRU
GARCH
Transformers

ABSTRACT

The non-linear and non-stationary nature of financial time series data poses significant challenges for standalone statistical and neural network methods. While predictive modeling in finance often focuses on volatility, there is a notable lack of research on predicting actual stock prices, particularly in the African market. This study addresses this gap by utilizing Airtel stock data from Yahoo Finance, spanning June 28, 2019, to May 8, 2024. The research employs the GARCH model to extract statistical properties, which are then combined with historical prices and fed into LSTM, GRU, and Transformer models leading to GARCH-LSTM, GARCH-GRU, GARCH-Transfomer hybrid models. These hybrid models are benchmarked against standalone LSTM, GRU and Transfomer models using RMSE, MAE, MAPE, and R-squared metrics. Results indicate that hybrid models, especially GARCH-LSTM, significantly outperform standalone models. This integration of GARCH with advanced AI models offers a more robust framework for stock price prediction, enhancing accuracy and reliability in forecasting future prices.

Introduction

In statistics, time series forecasting is widely used in various fields such as finance [1–3], climate science [4,5], and energy [6,7] among others [8]. Accurate time series prediction is challenging due to complex characteristics like seasonal patterns, trends, and volatility. Various methods, such as Autoregressive Integrated Moving Average (ARIMA) [9] and Generalized Autoregressive Conditional Heteroskedasticity (GARCH) [10], have been used to forecast univariate time series data. However, these methods mainly focus on linear trends, which do not always apply to real-world data that often exhibit nonlinear and nonstationary behaviors [11–14].

Recent advances in deep learning techniques have significantly improved time series forecasting. Artificial Neural Networks (ANNs) outperform traditional statistical methods in handling non-stationary and non-linear time series data [15–18]. Recurrent Neural Networks (RNNs) enhance ANN capabilities by maintaining a state between inputs, making them well-suited for time series forecasting [19–21]. Long Short-Term Memory (LSTM) networks address RNNs' issues, such as gradient vanishing and exploding, through three gate mechanisms [22], and have shown impressive results in time series forecasting [23–27]. Gated Recurrent Units (GRUs) further streamline LSTM by combining the three gates into two, often outperforming LSTM in various tasks [28–31]. Transformers, using an encoder–decoder structure and multi-head attention mechanism, have also gained attention for their ability to model long-term dependencies in time series data [32,33].

Despite the advances in statistical and deep learning methods, these approaches often struggle to capture the high volatility inherent in non-linear and non-stationary time series data. This limitation has led to growing interest in hybrid models, which

* Corresponding author.
E-mail addresses: jkmutinda@aimsammi.org (J.K. Mutinda), moskiplangat@gmail.com (A.K. Langat).

https://doi.org/10.1016/j.sciaf.2024.e02374
Received 4 June 2024; Received in revised form 13 July 2024; Accepted 13 September 2024
Available online 19 September 2024

Nomenclature

ACF	AutoCorrelation Function
AIC	Akaike Information Criterion
ANN	Artificial Neural Network
ARIMA	AutoRegressive Integrated Moving Average
EMD	Empirical Mode Decomposition
GARCH	Generalized Autoregressive Conditional Heteroskedasticity
GRU	Gated Recurrent Unit
IMF	Intrinsic Mode Function
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
PACF	Partial AutoCorrelation Function
RMSE	Root Mean Square Error
RNN	Recurrent Neural Network
VMD	Variational Mode Decomposition

combine the strengths of various techniques to handle the complexities of volatile time series data more effectively. Hybrid models offer the potential to improve forecasting accuracy by integrating the capabilities of both statistical and machine learning methods, thus addressing the shortcomings of each approach when used in isolation. This recognition drives the aim of our research to explore and develop hybrid models for more robust and accurate time series forecasting.

Research objectives

This research aims to develop an advanced hybrid forecasting model by integrating deep learning techniques with traditional statistical methods. The specific objectives are:

- Design a hybrid time series forecasting model using historical data, integrating statistical techniques and modern deep learning methods such as LSTM, GRU, and Transformers.
- Evaluate the performance of the proposed hybrid model against benchmark models to determine the trade-off between accuracy and complexity.
- Apply the hybrid model to predict financial time series data and present the results using visualization and performance metrics, including MAE, RMSE, MAPE, and R-squared.

Organization

This paper is organized as follows: Section “Related works” reviews related work and identifies the research gap. Section “Data and Methods” details the methodology, including data preprocessing, model development, implementation, and evaluation metrics. Section “Results and Discussion” presents the forecasting results. Finally, Section “Conclusion” concludes the study.

Related works

Combining multiple methods has become popular in time series forecasting, with the idea that it can improve prediction accuracy compared to using standalone statistical or AI methods. This approach, known as hybridization, is based on the divide-and-conquer strategy widely used in various fields like military tactics, business, and personal goal-setting [34]. The strategy involves breaking down a big problem into smaller, more manageable parts, which helps in focusing, organizing, and efficiently using resources.

In scientific research, particularly in time series modeling, this approach involves preprocessing data by decomposing it into several simpler subcomponents. Each subcomponent is easier to train using deep neural networks, and the forecasts from each are combined to provide the final prediction [35–37]. Decomposition methods in hybrid models are mainly of three types: wavelet transform (WT), empirical mode decomposition (EMD), and variational mode decomposition (VMD) [38]. WT requires manual selection of parameters, while EMD and its variant EEMD adaptively break down data based on local characteristics but can suffer from issues like spurious components. VMD effectively addresses these challenges and is especially good for handling complex, nonlinear, and non-stationary time series data [39,40].

Hybrid models using these decomposition methods have shown superior performance in many forecasting tasks, including wave forecasting [41–43], stock market index prediction [44], weather prediction [34,45], and traffic flow prediction [46].

Integrating statistical models with deep learning forecasters to create a hybrid forecasting model has also shown promising results. For example, [47] explored various techniques, including ARIMA and ANN, and hybrid approaches like ARIMA-ANN. Their results showed that the ARIMA-ANN hybrid consistently outperformed the individual models in stock market predictions.

Similarly, [48] addressed the limitations of single gas prediction models for mine gas concentration data by proposing an ARIMA-LSTM model. The ARIMA model handles linear predictions, while the LSTM model captures nonlinear factors. Their results showed the ARIMA-LSTM model provided higher accuracy.

In another study, [49] proposed a hybrid ARIMA-LSTM model for well production prediction, addressing the limitations of traditional models. This hybrid model captures both linear trends and nonlinear fluctuations, showing better performance than individual models.

Additionally, [50] proposed a novel hybrid model combining ARFIMA and LSTM to predict fast-fluctuating financial data. Their model effectively captured long-memory dynamics and nonlinearities, showing better performance compared to standalone models.

Recently, hybrid models combining GARCH-type models with ANN models have shown significant improvements in predicting financial time-series data. These models, termed ANN-GARCH hybrids, have been successful in forecasting various financial indicators like stock indices, gold prices, oil prices, and cryptocurrencies [51–55]. For example, [56] combined GARCH models with deep learning models to forecast cryptocurrency volatility, showing that hybrid models improve prediction accuracy.

However, most existing research has focused on volatility prediction, with fewer studies combining statistical and deep learning methods for actual financial time series prices. The integration of transformer models with econometric models also remains unexplored. This research aims to bridge these gaps by developing a hybrid model that integrates statistical methods with AI-based models, specifically GRU, LSTM, and transformer architectures, to predict actual stock prices.

Research gap

The application of hybrid models combining GARCH (Generalized Autoregressive Conditional Heteroskedasticity) with advanced AI methods like LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit) has predominantly focused on forecasting financial market volatility. While these approaches have demonstrated significant potential in capturing the complexities of volatility dynamics, their utilization in direct stock price forecasting remains limited.

Moreover, despite the growing interest in the transformative capabilities of AI, the combination of GARCH models with state-of-the-art transformer architectures has not been extensively explored. Transformers, known for their prowess in handling sequential data and capturing long-range dependencies, could potentially offer substantial improvements in forecasting accuracy when integrated with GARCH models. However, literature reveals a notable absence of studies that investigate this hybrid approach, leaving a critical gap in both theoretical and applied research.

This study seeks to bridge these gaps by developing and testing a hybrid model that integrates GARCH with LSTM, GRU and transformer architectures for stock price forecasting, with a particular focus on African markets. By addressing this unexplored intersection, the research aims to enhance forecasting accuracy and provide valuable insights into the dynamics of stock prices in emerging economies. .

Data and methods

Data and case study

The dataset used in this study includes stock data from Airtel, a leading telecommunications company in Africa, covering the period from June 28, 2019, to May 8, 2024. The data contains daily trading information from Monday to Friday, excluding holidays. The variables in the dataset are Open, High, Low, Close, Adjusted Close, and Volume. Although the dataset is comprehensive, it had one missing value, which was filled using Generative Adversarial Networks (GANs) to ensure data completeness and maintain the analysis's integrity.

We selected the Close price as the primary variable for analysis, following common practices in financial research [56–59]. The Close price is often used for computing technical indicators and forecasting time series.

The Generalized Autoregressive Conditional Heteroskedasticity (GARCH)

The GARCH model, introduced by Tim Bollerslev in 1986 [60], extends the ARCH model to offer a more flexible framework for modeling volatility clustering and persistence in financial time series data. The GARCH(p,q) model captures these features through a mean equation and a variance equation.

The mean equation is:

$$y_t = \mu + \epsilon_t \tag{1}$$

Here, y_t is the observed return at time t, μ is the mean return, and ϵ_t is the error term at time t. The error term ϵ_t is modeled as:

$$\epsilon_t = \sigma_t z_t \tag{2}$$

where σ_t is the conditional standard deviation (volatility) at time t, and z_t is a white noise error term with mean zero and variance one, typically assumed to be normally distributed ($z_t \sim N(0,1)$).

The variance equation is:

$$\sigma_t^2 = \alpha_0 + \sum_{i=1}^{q} \alpha_i \epsilon_{t-i}^2 + \sum_{j=1}^{p} \beta_j \sigma_{t-j}^2 \tag{3}$$

In this equation, σ_t^2 is the conditional variance at time t. The parameter α_0 is a constant term and must be positive ($\alpha_0 > 0$). The parameters α_i (where $i = 1, \ldots, q$) are associated with the lagged squared residuals, also known as ARCH terms, and must be non-negative ($\alpha_i \geq 0$). The parameters β_j (where $j = 1, \ldots, p$) are associated with the lagged conditional variances, known as GARCH terms, and must also be non-negative ($\beta_j \geq 0$). The integers p and q represent the number of lagged conditional variances and lagged squared residuals included in the model, respectively.

The parameters of the GARCH model ($\alpha_0, \alpha_i, \beta_j$) are typically estimated using Maximum Likelihood Estimation (MLE). MLE is preferred because it provides efficient and unbiased parameter estimates under the assumption that the underlying distribution, such as the normal distribution, holds true. The log-likelihood function for the GARCH model, assuming normally distributed errors, is given by:

$$\ln L(\theta) = -\frac{n}{2} \ln(2\pi) - \frac{1}{2} \sum_{t=1}^{n} \left[\ln(\sigma_t^2) + \frac{\epsilon_t^2}{\sigma_t^2} \right] \tag{4}$$

Here, θ represents the vector of parameters to be estimated ($\alpha_0, \alpha_1, \ldots, \alpha_q, \beta_1, \ldots, \beta_p$), and n is the number of observations.

GARCH models are widely used in financial econometrics because they effectively capture volatility clustering, a common phenomenon where large changes in financial returns tend to be followed by large changes (of either sign), and small changes tend to be followed by small changes. This property makes GARCH models valuable for risk management, option pricing, and financial forecasting. By accounting for time-varying volatility, GARCH models provide more accurate and reliable estimates of future volatility than models assuming constant volatility. While there are many GARCH variants, in this study we will use the standard GARCH with a t-distribution of the residuals.

Long short term memory

LSTM (Long Short-Term Memory) is a specialized type of recurrent neural network (RNN) designed to address the vanishing gradient problem commonly encountered in traditional RNNs, which hampers their ability to learn and retain information over long sequences. LSTM networks excel at modeling and predicting sequential data, making them ideal for tasks such as time series forecasting, natural language processing, and speech recognition [23–27].

An LSTM unit consists of several key components: a cell state C_t, which acts as the network's memory, and three gates that regulate the flow of information into and out of this cell state. These gates are the input gate i_t, the forget gate f_t, and the output gate o_t. The gates allow the LSTM to selectively update, retain, or discard information, enabling it to manage long-term dependencies effectively.

The behavior of an LSTM unit is governed by the following equations:

1. **Input Gate**: Determines how much of the new information to store in the cell state.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \tag{5}$$

2. **Forget Gate**: Decides how much of the previous cell state information to forget.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \tag{6}$$

3. **Output Gate**: Controls the output of the cell state to the hidden state.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \tag{7}$$

4. **Candidate Cell State**: Generates new candidate values that could be added to the cell state.

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \tag{8}$$

5. **Cell State Update**: Updates the cell state by combining the old state and the new candidate state.

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t \tag{9}$$

6. **Hidden State Update**: Updates the hidden state using the new cell state.

$$h_t = o_t \cdot \tanh(C_t) \tag{10}$$

where: i_t, f_t, and o_t are the input, forget, and output gates respectively, $\tilde{C}_t$ is the candidate cell state, C_t is the cell state, h_t is the hidden state, x_t is the input at time step t, W_i, W_f, W_o, W_c, b_i, b_f, b_o, b_c are the weight matrices and bias vectors of the gates, σ is the sigmoid activation function, and $[h_{t-1}, x_t]$ denotes concatenation. Fig. 1 shows the illustration of LSTM.

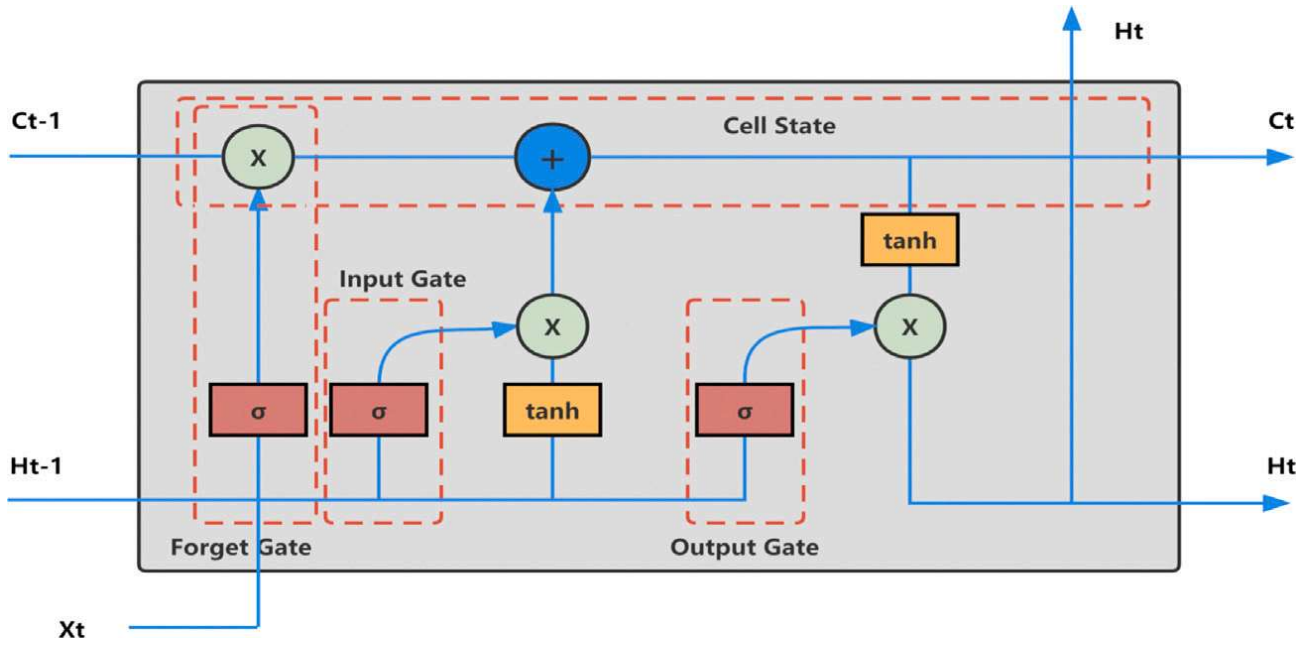

Fig. 1. An Illustration of LSTM.
Source: Source

Gated recurrent units

Gated Recurrent Unit (GRU) is a type of recurrent neural network (RNN) architecture designed to address limitations of traditional RNNs, such as difficulties in learning long-range dependencies and the vanishing gradient problem. GRU simplifies the architecture of the Long Short-Term Memory (LSTM) unit while retaining its ability to capture long-term dependencies in sequential data [29–31,61–63].

The behavior of a GRU unit is governed by several key equations. The first component is the update gate, z_t, which determines how much of the past information needs to be passed along to the future. This gate is calculated as follows:

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t]) \tag{11}$$

In this equation, z_t is the update gate vector, W_z is the weight matrix for the update gate, $[h_{t-1}, x_t]$ represents the concatenation of the previous hidden state h_{t-1} and the current input x_t, and σ is the sigmoid activation function.

Next, the reset gate, r_t, decides how much of the past information to forget. This is calculated using:

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t]) \tag{12}$$

Here, r_t is the reset gate vector, W_r is the weight matrix for the reset gate, and $[h_{t-1}, x_t]$ again represents the concatenation of h_{t-1} and x_t, with σ being the sigmoid activation function.

The candidate hidden state, $\tilde{h}_t$, is computed considering the reset gate's filtering of past information. This is given by:

$$\tilde{h}_t = \tanh(W \cdot [r_t \cdot h_{t-1}, x_t]) \tag{13}$$

In this equation, $\tilde{h}_t$ is the candidate hidden state vector, W is the weight matrix, and $r_t \cdot h_{t-1}$ represents the element-wise multiplication of the reset gate r_t and the previous hidden state h_{t-1}. The activation function tanh introduces non-linearity to model more complex relationships.

Finally, the hidden state h_t is updated by combining the previous hidden state h_{t-1} and the candidate hidden state $\tilde{h}_t$, modulated by the update gate z_t:

$$h_t = (1 - z_t) \cdot h_{t-1} + z_t \cdot \tilde{h}_t \tag{14}$$

Here, h_t is the new hidden state. The terms $1 - z_t$ and z_t control the contributions of the previous hidden state and the candidate hidden state to the new hidden state, respectively. This update mechanism allows the GRU to adaptively remember or forget past information, effectively capturing long-term dependencies in the data. Fig. 2 shows illustration of GRU.

Transfomers

Transformers have revolutionized various fields of artificial intelligence, including natural language processing, image recognition, and more recently, time series analysis. When adapted for time series data, transformers offer a powerful framework for capturing complex temporal dependencies and making accurate forecasts. At the core of the transformer architecture are encoder and decoder layers, each equipped with self-attention mechanisms that enable the model to efficiently process sequential data [32,33,64]

In the context of time series analysis, the encoder receives input sequences representing historical observations. Each input sequence is passed through a stack of encoder layers, with each layer performing operations such as self-attention and feedforward neural networks. The self-attention mechanism allows the encoder to capture dependencies between different time steps in the input sequence, enabling it to identify patterns and extract meaningful features from the data. In transformers, the self-attention

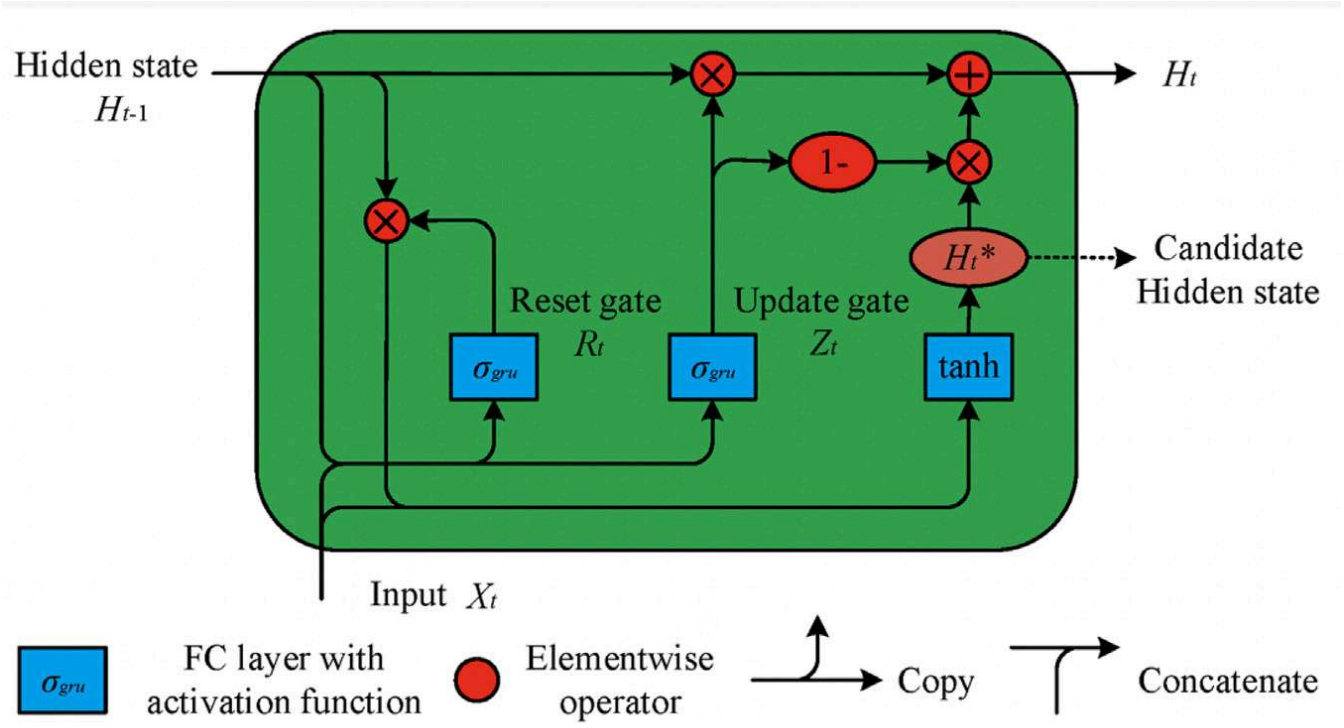

Fig. 2. An illustration of GRU.
Source: Source

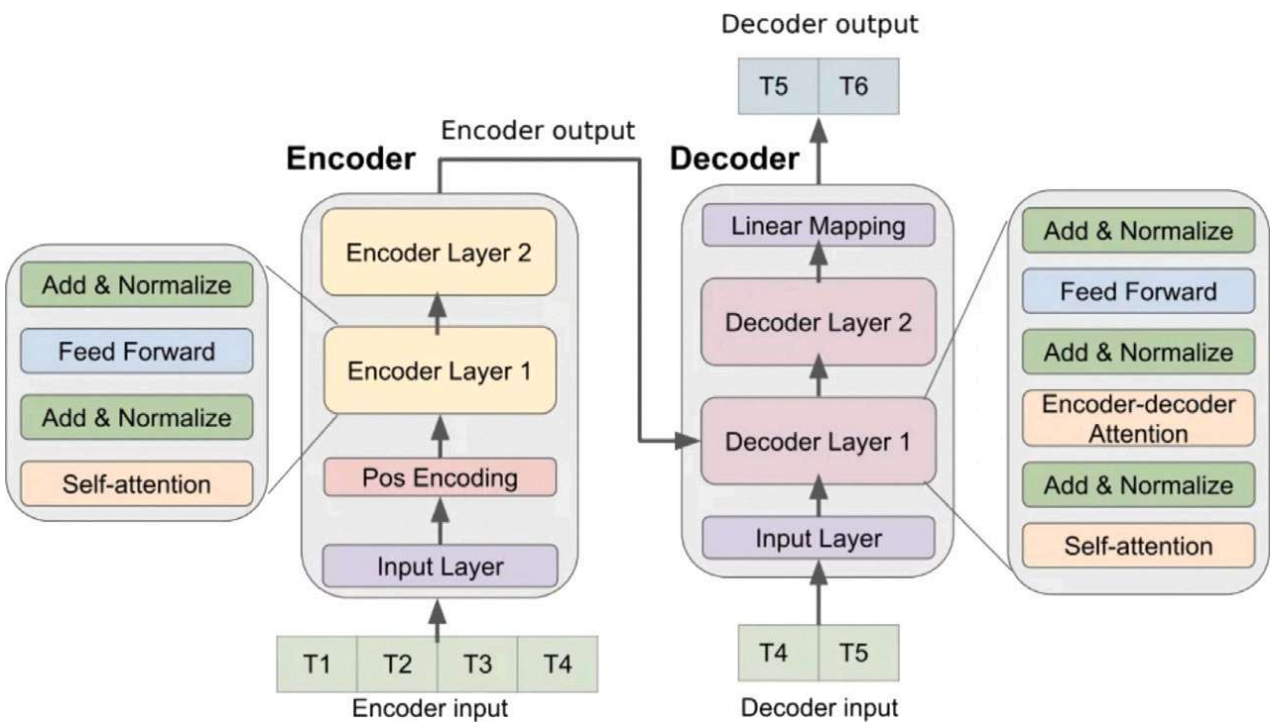

Fig. 3. An illustration of transformer based model.
Source: Source

mechanism allows each input token to attend to all other tokens to capture interdependencies. The self-attention score between tokens i and j is computed a

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V} \tag{15}$$

where **Q**, **K** and **V** are query, key, and value matrices, respectively, and is the dimension of the key vectors. Additionally, positional encoding is added to the input embeddings to provide the model with information about the order of the data points in the sequence. One common way of positioning encoding is by use the use of sinusoidal functions as show below Once the input sequence has been processed by the encoder, the resulting representations are passed to the decoder. In single-output forecasting tasks, where the goal is to predict a single future time step, the decoder generates predictions based on the encoded representations of the input sequence. The decoder also incorporates a self-attention mechanism to capture dependencies between different elements of the output sequence and a cross-attention mechanism to attend to relevant parts of the input sequence. Finally, the output of the decoder is passed through a linear layer to produce the predicted value for the next time step. Fig. 5 shows an illustration of transformer based model (see Figs. 3 and 4).

Fig. 4. Time series plot of daily closing price of airtel stock.

Experimental design and parameter settings

In our experiment, we began by computing the logarithmic returns from the stock price data. The formula used for the logarithmic return r_t is:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$$

where P_t is the closing price at time t and P_{t-1} is the closing price at time $t-1$. This logarithmic return was then used to extract the conditional volatility through the GARCH model, which served as an input for the LSTM, GRU, and Transformer models, along with the historical stock prices.

For the training and testing process, 67% of the data was used for training, while the remaining 33% was reserved for testing. This split ensured that the models had sufficient data for learning while also providing a robust test set for evaluating performance. The hybrid models developed were GARCH-LSTM, GARCH-GRU, and GARCH-Transformer, which were compared against standalone LSTM, GRU, and Transformer models using metrics such as RMSE (Root Mean Squared Error), MAE (Mean Absolute Error), MAPE (Mean Absolute Percentage Error), and R-squared.

To ensure a fair comparison, the parameter settings for LSTM and GRU were standardized. A grid search was conducted for LSTM, considering two layers each with 100, 50, 128, and 64 units, batch sizes of 64 and 32, and epochs of 50 and 100. The learning rates of 0.001 and 0.0001 were tested using Adam and RMSprop optimizers, with a dropout rate of 0.2 to prevent overfitting. The optimal parameters identified for LSTM were 128 and 64 units with a batch size of 32, 50 epochs, and a learning rate of 0.0001 using the Adam optimizer and the loss function to be minimized was set as mean square logarithmic loss. These parameters were fixed for GRU, GARCH-GRU, and GARCH-LSTM to maintain consistency across comparisons.

For the Transformer model, similar rigorous tuning was applied. The Transformer architecture was defined with an input dimension of 1 for Transfomer and 2 for GARCH-Transfomer, a model dimension (d_{model}) of 64, 4 attention heads, and 2 layers, with a dropout rate of 0.2. The training used the Adam optimizer with a learning rate of 0.0001 and a learning rate scheduler (ReduceLROnPlateau) to adjust the learning rate based on validation loss. The loss function was set as mean square logarithmic loss. The training was conducted for up to 10,000 epochs, with early stopping applied if the validation loss did not improve for 5 consecutive epochs. This setup ensured that the Transformer models, both standalone and GARCH-Transformer hybrids, were optimized for performance. All the analysis was done using Python.

Evaluation metrics

The root mean square error, mean absolute error, mean absolute percentage error and r squared performance evaluation metrics where used to quantify the model performance .

Root mean squared error (RMSE)

RMSE measures the average magnitude of the errors between predicted values and observed values in a regression model. It is defined as:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

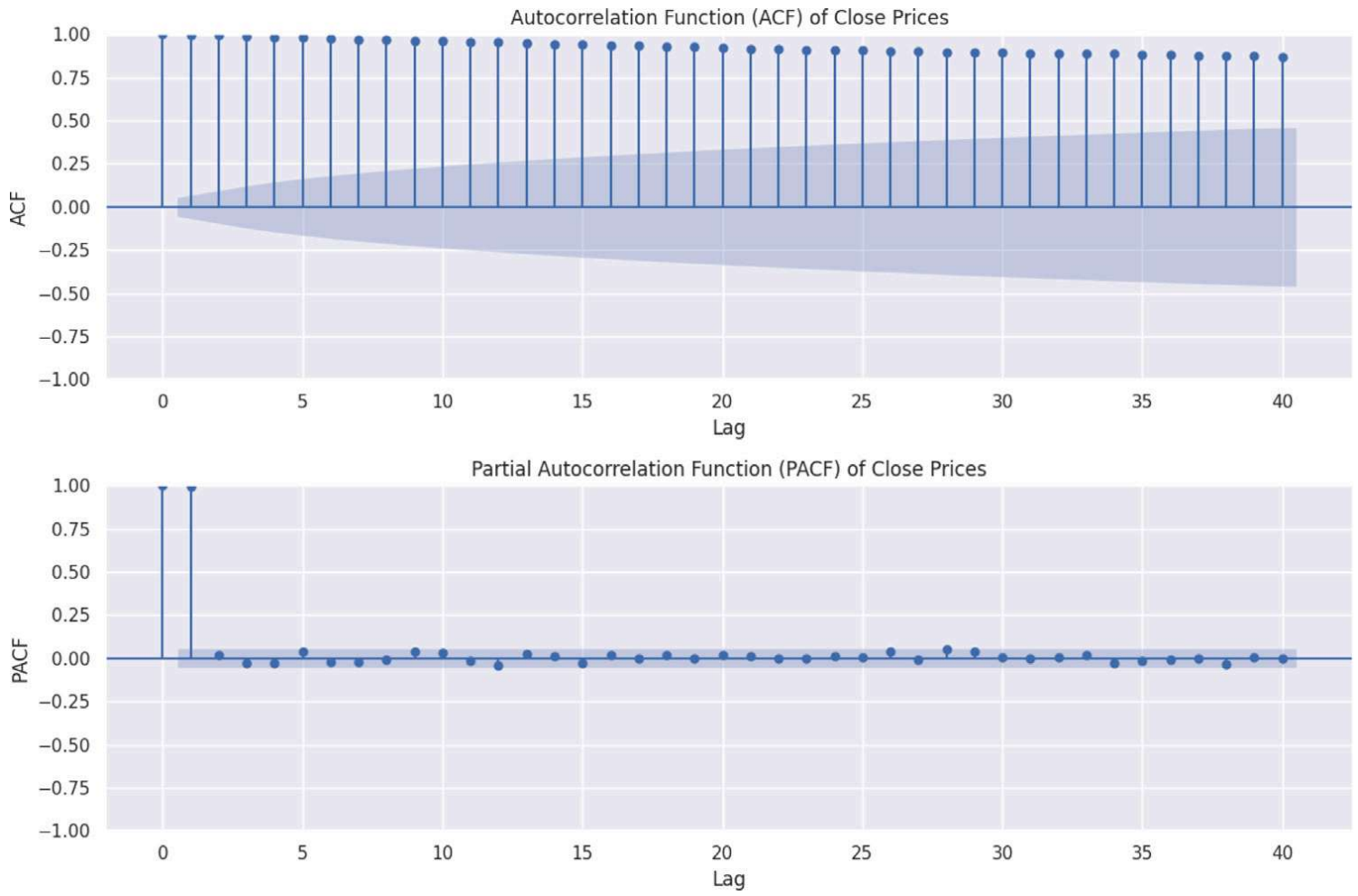

Fig. 5. ACF and PACF plot for close price.

where n is the number of observations, y_i is the observed value, and $\hat{y}_i$ is the predicted value.

Mean absolute error (MAE)

MAE also measures the average magnitude of the errors, but it takes the absolute value of the errors instead of squaring them. It is defined as:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Mean absolute percentage error (MAPE)

MAPE measures the percentage difference between predicted and observed values. It is defined as:

$$\text{MAPE} = \frac{1}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right| \times 100\%$$

Coefficient of determination (R-squared)

R-squared measures the proportion of the variance in the dependent variable that is predictable from the independent variables. It is defined as:

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

where $\bar{y}$ is the mean of the observed values.

Results and discussion

Stationarity and non linearity test

Fig. 5 shows a time series plot of the daily stock prices of the Airtel stock, by eye inspection, the series is non linear and non stationary. However, eye inspection is not sufficient to validate this conditions. A stationarity test was conducted using the ADF test at 5% level of significance. The ADF test for close prices, with a statistic of −1.562 and a p-value of 0.502, fails to reject the null hypothesis that the series has a unit root at the 5% significance level. This indicates that the close prices are non-stationary. To assert non linearity in the data, the ACF and PACF plot of the time series was plotted. Figure below show the ACF and PACF plot of the close price series. It can be concluded from the ACF plot that most of the lags spike above the significant bound suggesting the persisting non linearity in the series. A Ljung box test statistic was conducted at 5% level of significance. The Ljung–Box test for close prices, with a statistic of 11798.71 and a p-value of 0.0, rejects the null hypothesis of no autocorrelation at the 5% significance

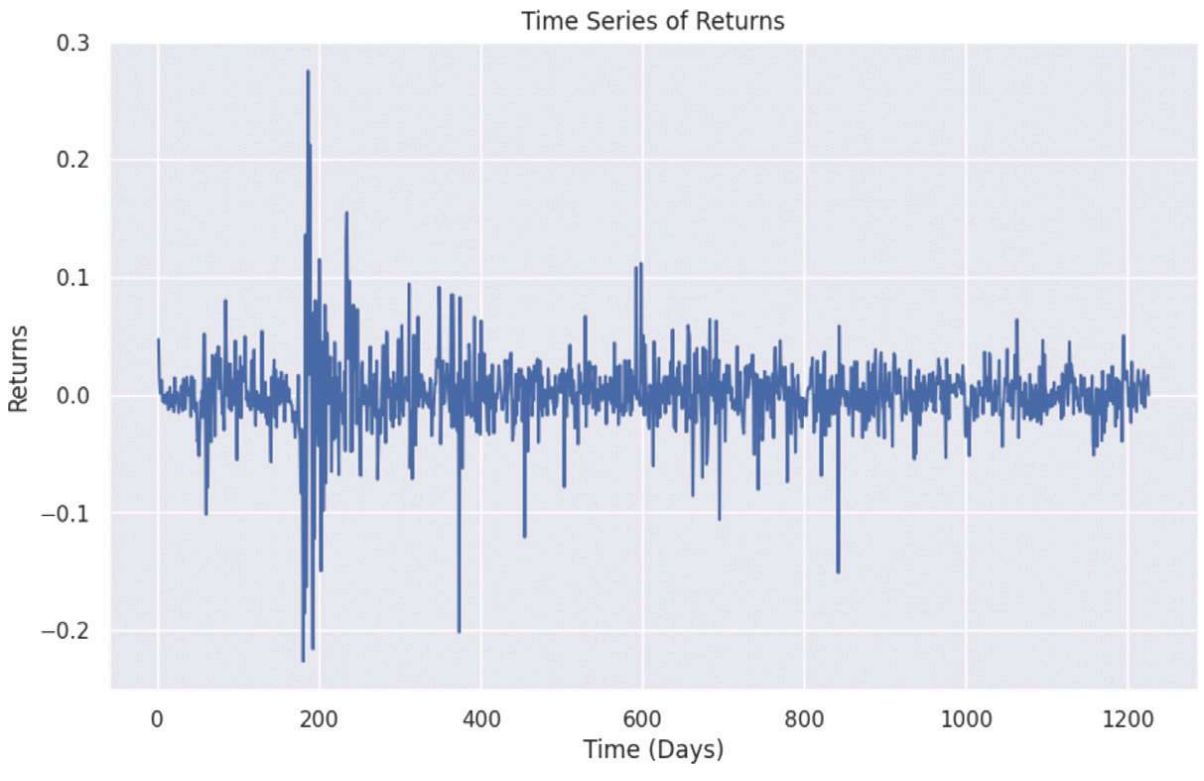

Fig. 6. ACF and PACF plot for close price.

level. This indicates that there is significant autocorrelation in the close prices. The presence of autocorrelation suggests that the values are not independent over time, which is important for time series modeling and forecasting. The ARCH test for close prices, with a statistic of 1204.96 and a p-value of approximately 0.0, rejects the null hypothesis of no ARCH effects at the 5% significance level. This indicates the presence of autoregressive conditional heteroskedasticity in the close prices. Significant ARCH effects imply that the volatility of the series changes over time.

GARCH model validity

As we have asserted that the time series is not stationary, this is not sufficient for development of the GARCH model for the series. We therefore inspect the statistical behavior of the returns. From Fig. 7 it is observed that the returns tend to show stationary characteristics as the series is tend to move in a constant mean. However this need to be asserted by an ADF test. The ADF test for returns, with a statistic of −10.38 and a p-value of approximately 0.0, rejects the null hypothesis of a unit root at the 5% significance level. This indicates that the returns series is stationary. Stationarity suggests that the statistical properties of the returns, such as the mean and variance, are constant over time. The ARCH test for returns, with a statistic of 436.86 and a p-value of approximately 0.0, rejects the null hypothesis of no ARCH effects at the 5% significance level. This indicates that there is significant autoregressive conditional heteroskedasticity in the returns series. The presence of ARCH effects means that the volatility of returns varies over time. We conclude that the returns are stationary and they also portray autoregressive conditional heteroskedasticity. This imply that GARCH model can be used to extract the statistical properties of the series (see Figs. 6 and 10).

GARCH fitting and extraction of statistical properties

After verification that a GARCH model is suitable for the series. We fitted a standard GARCH model with a student t distribution of the residues to our time series. To fit a standard GARCH model on the returns, we performed a grid search over possible orders (p, q) ranging from 1 to 3. The optimal parameters were selected based on the Akaike Information Criterion (AIC), aiming for the model with the lowest AIC, indicating the best fit. Below is a table showing all possible GARCH (p, q) combinations and their corresponding AIC values (see Table 1). Based on AIC, GARCH(1,1) is the best and it is chosen to model the returns. The focus is now to exact the conditional volatility which is used as an input together with the historical price to predict future closing price of the Airtel stock.

The statistical results of GARCH (1,1) are presented as shown below

The results of the GARCH(1,1) model fitting for the returns series are presented in Table 2. The omega parameter, which represents the constant term, has a coefficient of 1.9818×10^{-5} with a very low standard error, resulting in a highly significant t-statistic of 11.635 and a p-value close to zero, indicating its strong statistical significance. The alpha parameter, corresponding to the lagged squared return, has a coefficient of 0.1000 with a higher standard error and a t-statistic of 1.323, leading to a p-value of 0.186, which suggests that alpha is not statistically significant at the 5% level. The beta parameter, representing the lagged conditional variance, has a coefficient of 0.8800 with a low standard error and a very high t-statistic of 17.262, making it highly significant with a p-value close to zero. The 95% confidence intervals for omega, alpha, and beta are $[1.648 \times 10^{-5}, 2.316 \times 10^{-5}]$, $[-0.04814, 0.248]$, and $[0.780, 0.980]$ respectively. These results indicate that while the volatility clustering effect is well captured by the significant beta parameter, the alpha parameter is not significant, suggesting that past squared returns do not have a substantial impact on current volatility in this model. The high significance of omega and beta underscores the model's ability to effectively

Table 1
AIC values for different GARCH (p, q) models.

(p, q)	AIC
(1, 1)	−5708.864994
(1, 2)	−5699.616976
(1, 3)	−5697.582289
(2, 1)	−5699.663811
(2, 2)	−5697.559413
(2, 3)	−5694.579648
(3, 1)	−5689.483542
(3, 2)	−5684.742794
(3, 3)	−5680.824510

Table 2
GARCH(1,1) model estimates.

Parameter	Coefficient	Std. Error	*t*-Statistic	P-Value	95% Conf. Int.
Omega	1.9818e−05	1.703e−06	11.635	2.752e−31	[1.648e−05, 2.316e−05]
Alpha	0.1000	7.558e−02	1.323	0.00186	[−4.814e−02, 0.248]
Beta	0.8800	5.098e−02	17.262	9.107e−67	[0.780, 0.980]

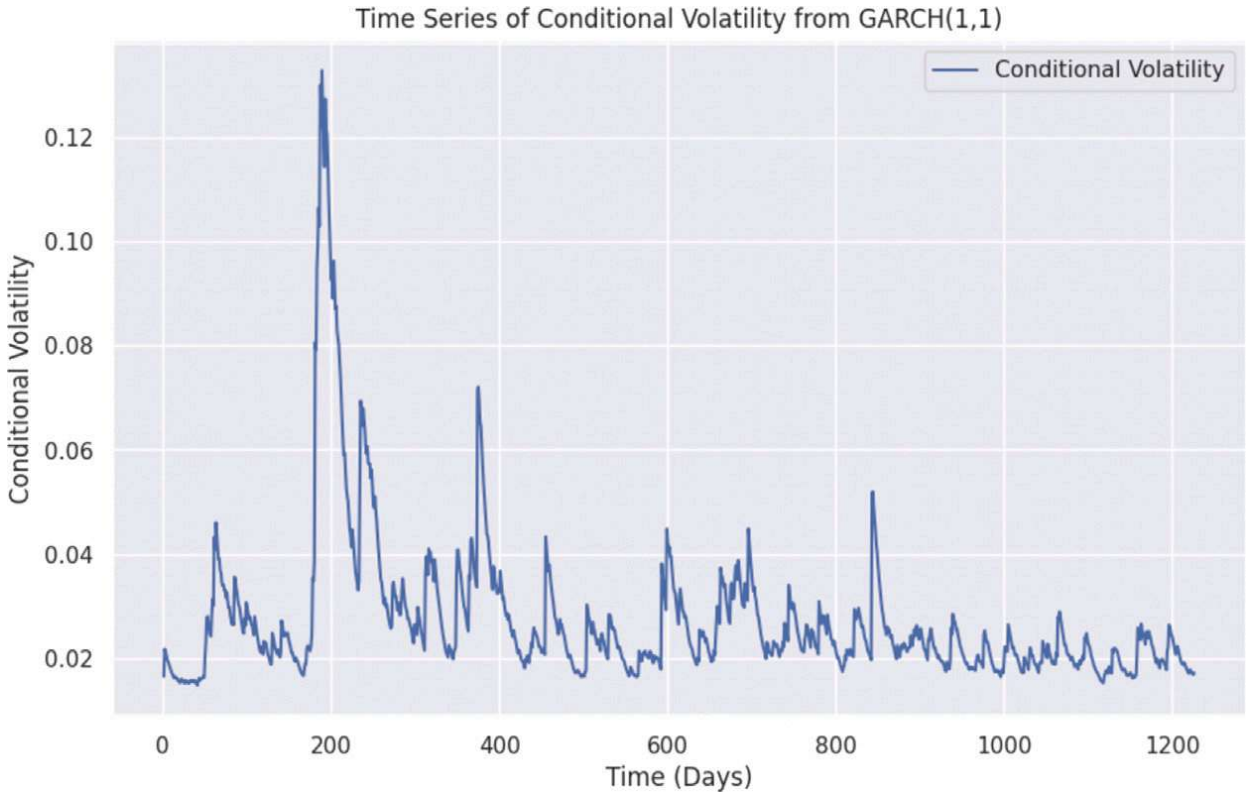

Fig. 7. Time series plot of daily conditional volatility.

capture the persistence in volatility. Figure below shows the plot of the conditional volatility extracted from GARCH(1,1) model. The extracted volatility which represent statistical properties of daily close price together with the historical daily closing prices were used as an input to the LSTM, GRU and Transfomer. This integration of GARCH and LSTM, GRU and Transfomer leads to GARCH-LSTM, GARCH-GRU and GARCH-Transfomer hybrid models. To compare that the hybrid models are able to achieve a better predictive performance, we set LSTM, GRU and Transfomer as our benchmark models. These hybrid models are hypothesized to perform better than the standalone statistical and deep learning methods.

Experimental results

The plots of the predictions obtained from GARCH-LSTM, LSTM and the observed daily close price for the out of sample data is as shown in Fig. 8. It can be observed keenly that the performance of the hybrid model surpasses that of the LSTM model when applied directly to historical prices. We observe that the forecasted align more accurately with actual value. This alignment is reflected in both magnitude and direction. In summary, there is only a small quantitative difference between the forecasted and actual values, but more consistent upward and downward trend at each time point.

Fig. 9 show the plot of predictions obtained from GARCH-GRU, GRU and the observed daily close price for the out of sample data. We observe that forecasts from the hybrid model align close with the actual close price interms of magnitude and direction. While interms of direction GRU also show similar results like GARCH-GRU, interms of magnitude, there is observable difference between predicted and the actual close price. This shows that the hybrid model performed better that the standalone GRU model.

Fig. 11 show the plot of predictions obtained from GARCH-Transfomer, Transfomer and the observed daily close price for the out of sample data. We observe that forecasts from the hybrid model align close with the actual close price interms of magnitude

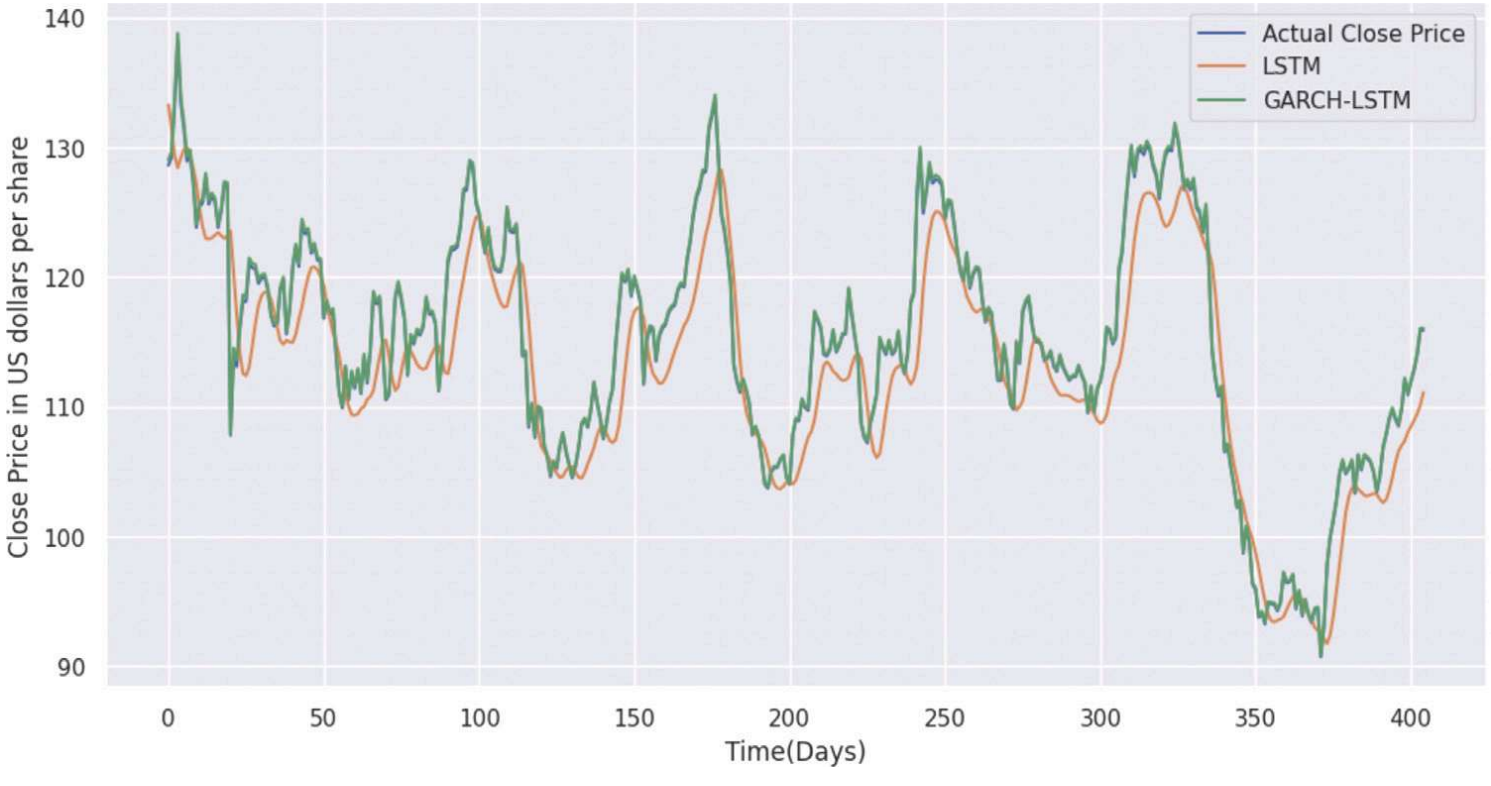

Fig. 8. Actual, LSTM predicted, GARCH-LSTM predicted of out of sample daily close price.

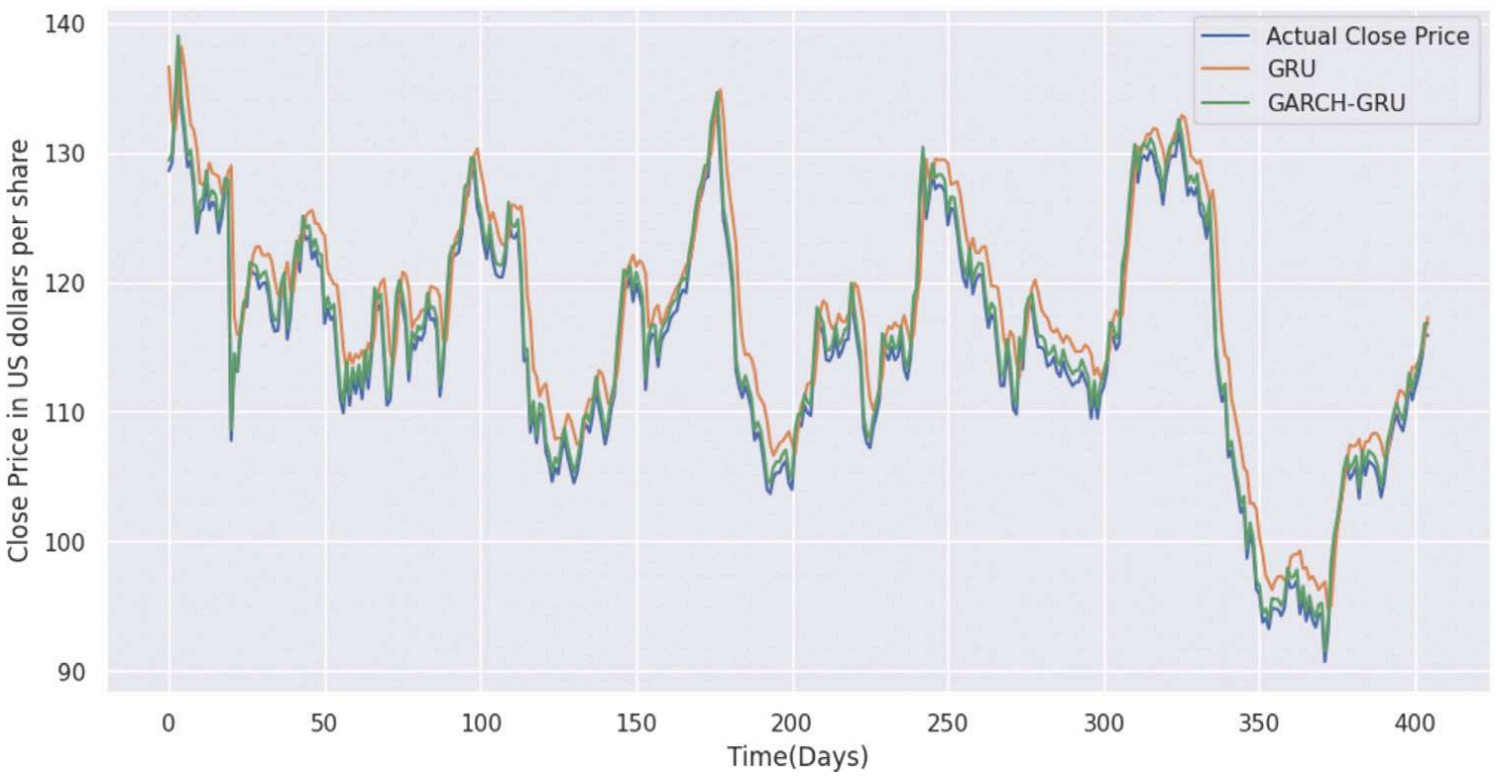

Fig. 9. Actual, GRU predicted, GARCH-GRU predicted of out of sample daily close price.

and direction. While in terms of direction GRU also show similar results like GARCH-Transformer, interms of magnitude, there is observable difference between predicted and the actual close price. This shows that the hybrid model performed better that the standalone Transfomer model.

Comparative analysis of all model results interms of RMSE, MAE, MAPE and R-squared

The Table 3 shows the performance metrics (RMSE, MAE, MAPE, and R-squared) for various forecasting methods, including standalone models and hybrid approaches. GARCH-LSTM demonstrates the best performance with the lowest RMSE (0.2002), MAE (0.1840), and MAPE (0.1570), and the highest R-squared (0.9995). Generally the hybrid models that combines GARCH with LSTM, GRU and Transfomer shows superior performance compared to the standalone LSTM, GRU and Transfomer models. This shows that integrating statistical properties of the historical prices with RNN and Transfomer is deemed to improve prediction performance of daily close prices of the Airtel Stock. By leveraging econometrics or statistical models with RNN and Transfomer to optimize forecast performance, this time series forecasting framework produces superior forecasts for the out of sample set. A comparative bar plot for the performance of the model is shown in figure

Conclusion

While most hybrid models focus on predicting technical indicators like volatility, this research uniquely applied a hybrid modeling approach to forecast the daily closing prices of Airtel stock, an Africa-based investment portfolio. By combining statistical

Table 3
Performance metrics for various methods.

Method	RMSE	MAE	MAPE	R-squared
GARCH-GRU	0.8482	0.8436	0.7347	0.9913
GRU	3.4567	2.7625	2.4354	0.8548
GARCH-LSTM	**0.2002**	**0.1840**	**0.1570**	**0.9995**
LSTM	3.9749	3.1643	2.7178	0.8079
GARCH-Transformer	1.6779	1.6773	1.4645	0.9658
Transformer	2.5303	1.8047	1.5696	0.9222

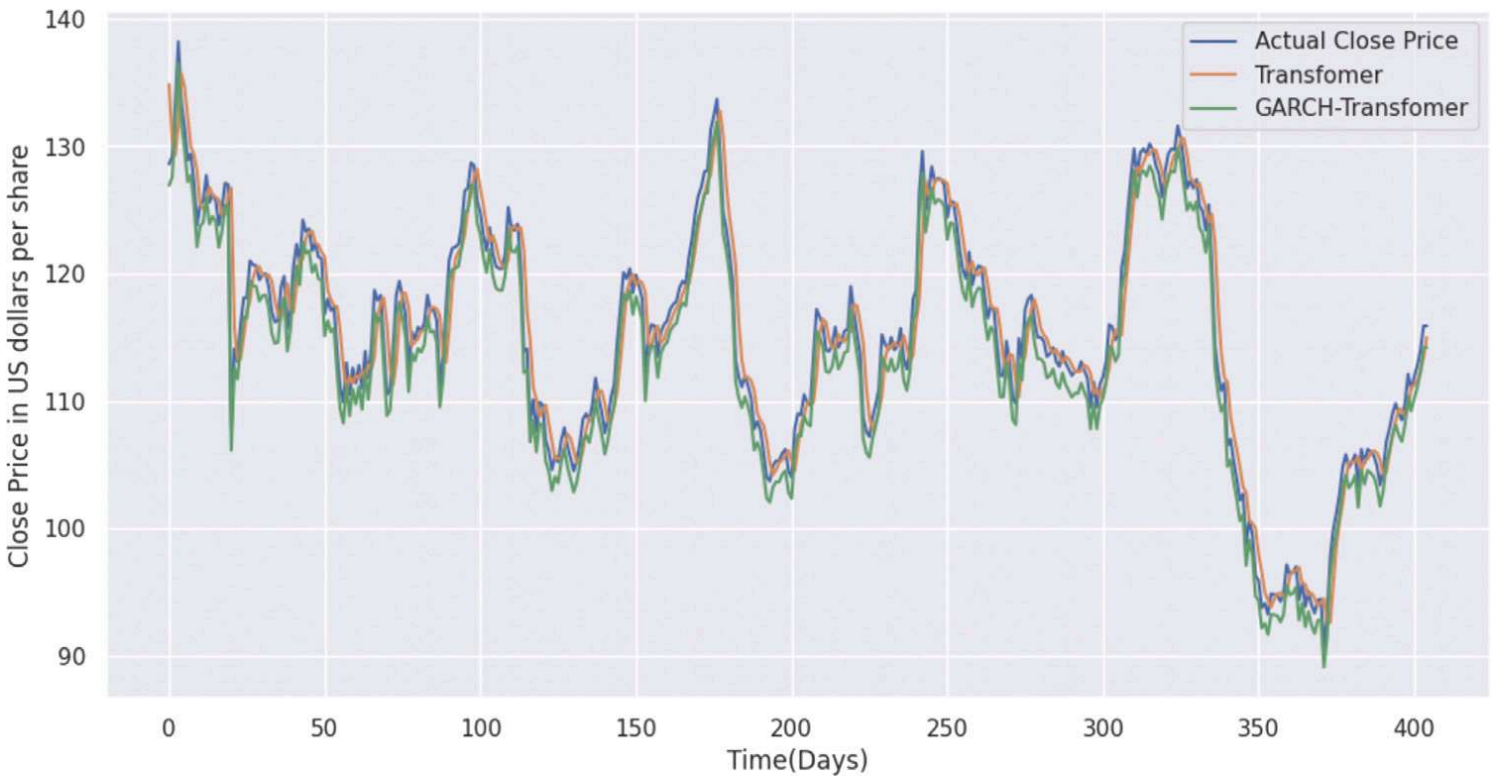

Fig. 10. Actual, Transfomer predicted, GARCH-Transfomer predicted of out of sample daily close price.

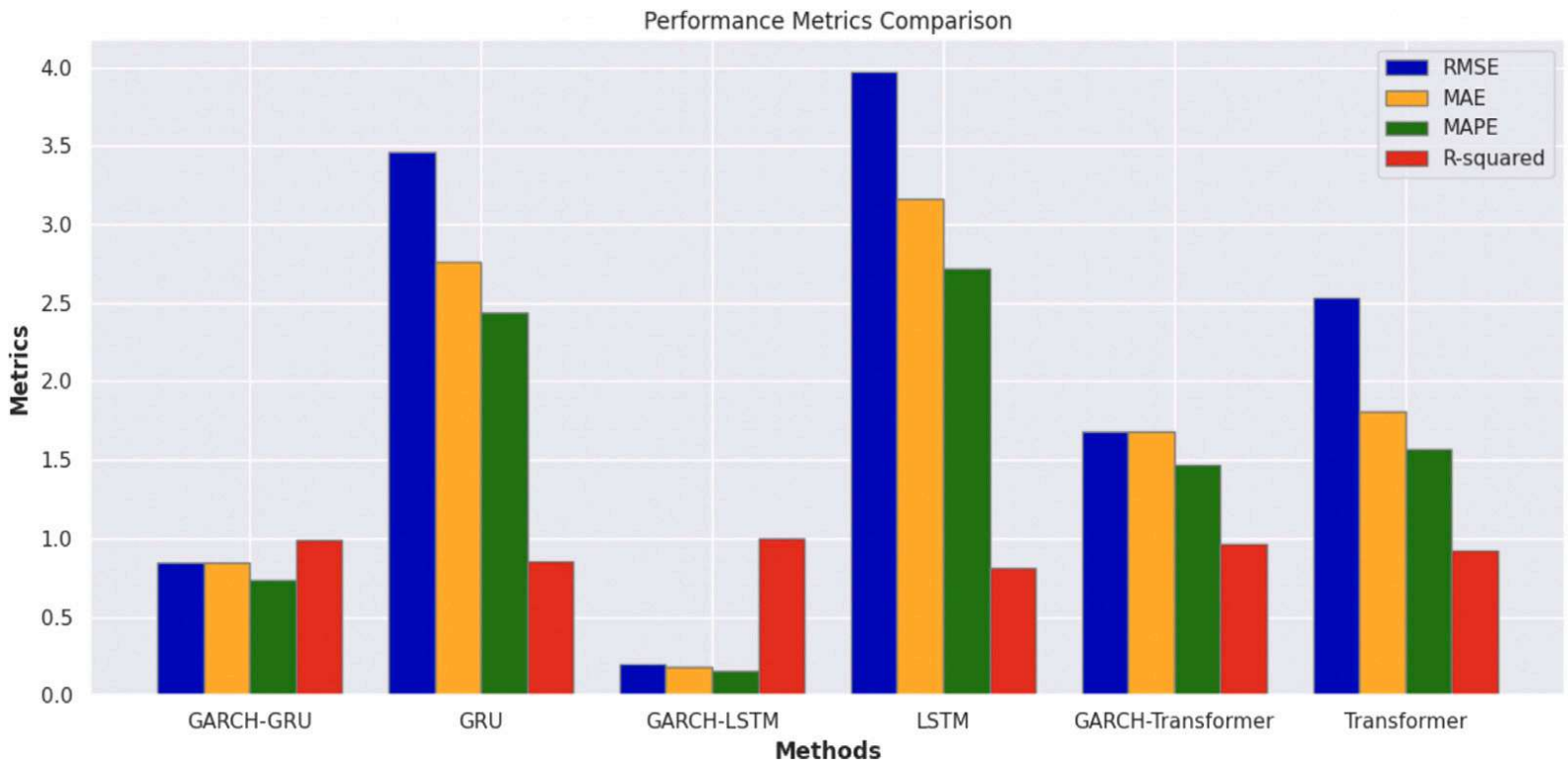

Fig. 11. Comparative analysis of forecasting performance of GARCH-GRU, GRU, GARCH-LSTM, LSTM, GARCH-Transfomer, Transfomer considering RMSE, MAE,MAPE and R-squared.

models like GARCH to capture the statistical properties of the data with RNNs and Transformers to model long-term dependencies, this study leveraged the strengths of both methodologies. This approach significantly improved forecasting accuracy compared to using standalone models.

The results demonstrate that the hybrid models, specifically GARCH-LSTM, GARCH-GRU, and GARCH-Transformer, consistently outperformed their standalone counterparts in terms of RMSE, MAE, MAPE, and R-squared metrics. The GARCH-LSTM model showed the best performance, highlighting the efficacy of integrating statistical properties of the historical data with advanced deep learning techniques. The superior performance of these hybrid models underscores the importance of addressing both the statistical properties and the temporal dependencies inherent in financial time series data.

This research illustrates the potential of hybrid models in enhancing stock price forecasting. By effectively combining econometric models with RNN and Transformer architectures, the study provides a framework for improving forecast accuracy in financial time series analysis. This hybrid approach not only offers better predictive performance but also opens new avenues for integrating various modeling techniques to tackle complex forecasting challenges in the financial domain. However, there are opportunities for further enhancements. These include optimizing model parameters, integrating more data sources, investigating ensemble methods, applying regularization techniques, improving evaluation metrics, testing different volatility models, and extending the forecasting period. These recommendations aim to boost the robustness, accuracy, and practical relevance of the models in real-world financial forecasting situations.

Funding

This research received no external funding.

CRediT authorship contribution statement

John Kamwele Mutinda: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization. **Amos Kipkorir Langat:** Conceptualization, Methodology, Validation, Resources, Writing – review & editing, Supervision, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data and Implementation materials are available upon request.

References

[1] Ngai Hang Chan, Time Series: Applications to Finance, John Wiley & Sons, 2004.
[2] Philip Hans Franses, Dick Van Dijk, Non-Linear Time Series Models in Empirical Finance, Cambridge University Press, 2000.
[3] John H. Cochrane, Time series for macroeconomics and finance, 1997.
[4] Jonas Knape, Perry de Valpine, Effects of weather and climate on the dynamics of animal population time series, Proc. R. Soc. B 278 (1708) (2011) 985–992.
[5] Manfred Mudelsee, Trend analysis of climate time series: A review of methods, Earth-Sci. Rev. 190 (2019) 310–322.
[6] Mahmoud Ghofrani, Musaad Alolayan, Time Series and Renewable Energy Forecasting, Vol. 10, IntechOpen, 2018.
[7] Fatih Onur Hocaoglu, Fatih Karanfil, A time series-based approach for renewable energy modeling, Renew. Sustain. Energy Rev. 28 (2013) 204–214.
[8] Peter Bidyuk, Aleksandr Gozhyj, Irina Kalinina, Victoria Vysotska, Methods for forecasting nonlinear non-stationary processes in machine learning, in: International Conference on Data Stream Mining and Processing, Springer, 2020, pp. 470–485.
[9] George E.P. Box, David A. Pierce, Distribution of residual autocorrelations in autoregressive-integrated moving average time series models, J. Am. Stat. Assoc. 65 (332) (1970) 1509–1526.
[10] Tim Bollerslev, A conditionally heteroskedastic time series model for speculative prices and rates of return, Rev. Econ. Stat. (1987) 542–547.
[11] Arthur Stepchenko, Jurij Chizhov, Ludmila Aleksejeva, Juri Tolujew, Nonlinear, non-stationary and seasonal time series forecasting using different methods coupled with data preprocessing, Procedia Comput. Sci. 104 (2017) 578–585.
[12] Changqing Cheng, Akkarapol Sa-Ngasoongsong, Omer Beyca, Trung Le, Hui Yang, Zhenyu Kong, Satish TS Bukkapatnam, Time series forecasting for nonlinear and non-stationary processes: a review and comparative study, Iie Trans. 47 (10) (2015) 1053–1071.
[13] Huimin Han, Zehua Liu, Mauricio Barrios Barrios, Jiuhao Li, Zhixiong Zeng, Nadia Sarhan, Emad Mahrous Awwad, Time series forecasting model for non-stationary series pattern extraction using deep learning and GARCH modeling, J. Cloud Comput. 13 (1) (2024) 2.
[14] Yann LeCun, Yoshua Bengio, Geoffrey Hinton, Deep learning, Nature 521 (7553) (2015) 436–444.
[15] Zhang Chengzhao, Pan Heping, Ma Yu, Huang Xun, Analysis of Asia Pacific stock markets with a novel multiscale model, Phys. A 534 (2019) 120939.
[16] Ana L.D. Loureiro, Vera L. Miguéis, Lucas F.M. Da Silva, Exploring the use of deep neural networks for sales forecasting in fashion retail, Decis. Support Syst. 114 (2018) 81–93.
[17] Jie Wang, Jun Wang, Forecasting stock market indexes using principle component analysis and stochastic time effective neural networks, Neurocomputing 156 (2015) 68–78.
[18] Yumo Xu, Shay B. Cohen, Stock movement prediction from tweets and historical prices, in: Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2018, pp. 1970–1979.
[19] Zhuang Zheng, Hainan Chen, Xiaowei Luo, Spatial granularity analysis on electricity consumption prediction using LSTM recurrent neural network, Energy Procedia 158 (2019) 2713–2718.
[20] Muskaan Pirani, Paurav Thakkar, Pranay Jivrani, Mohammed Husain Bohara, Dweepna Garg, A comparative analysis of ARIMA, GRU, LSTM and BiLSTM on financial time series forecasting, in: 2022 IEEE International Conference on Distributed Computing and Electrical Circuits and Electronics, ICDCECE, IEEE, 2022, pp. 1–6.
[21] Ruishuang QI, Lili DONG, Financial Time Series Forecasting Algorithm Based on Recurrent Neural Network.
[22] Sepp Hochreiter, Jürgen Schmidhuber, Long short-term memory, Neural Comput. 9 (8) (1997) 1735–1780.
[23] Kejun Wang, Xiaoxia Qi, Hongda Liu, Photovoltaic power forecasting based LSTM-convolutional network, Energy 189 (2019) 116225.
[24] Zahra Karevan, Johan A.K. Suykens, Transductive LSTM for time-series prediction: An application to weather forecasting, Neural Netw. 125 (2020) 1–9.
[25] Sepp Hochreiter, Jürgen Schmidhuber, Long short-term memory, Neural Comput. 9 (8) (1997) 1735–1780.

[26] Huaiyu Wan, Shengnan Guo, Kang Yin, Xiaohui Liang, Youfang Lin, CTS-LSTM: LSTM-based neural networks for correlatedtime series prediction, Knowl.-Based Syst. 191 (2020) 105239.
[27] Rajni Bala, Ram Pal Singh, et al., Financial and non-stationary time series forecasting using LSTM recurrent neural network for short and long horizon, in: 2019 10th International Conference on Computing, Communication and Networking Technologies, ICCCNT, IEEE, 2019, pp. 1–7.
[28] Dongxiao Niu, Min Yu, Lijie Sun, Tian Gao, Keke Wang, Short-term multi-energy load forecasting for integrated energy systems based on CNN-BiGRU optimized by attention mechanism, Appl. Energy 313 (2022) 118801.
[29] Zhiyun Peng, Sui Peng, Lidan Fu, Binchun Lu, Junjie Tang, Ke Wang, Wenyuan Li, A novel deep learning ensemble model with data denoising for short-term wind speed forecasting, Energy Convers. Manage. 207 (2020) 112524.
[30] Wenyi Wu, Wenlong Liao, Jian Miao, Guoli Du, Using gated recurrent unit network to forecast short-term load considering impact of electricity price, Energy Procedia 158 (2019) 3369–3374.
[31] Jimeng Shi, Mahek Jain, Giri Narasimhan, Time series forecasting (tsf) using various deep learning models, 2022, arXiv preprint arXiv:2204.11115.
[32] Qingsong Wen, Tian Zhou, Chaoli Zhang, Weiqi Chen, Ziqing Ma, Junchi Yan, Liang Sun, Transformers in time series: A survey, 2022, arXiv preprint arXiv:2202.07125.
[33] Sabeen Ahmed, Ian E Nielsen, Aakash Tripathi, Shamoon Siddiqui, Ravi P Ramachandran, Ghulam Rasool, Transformers in time-series analysis: A tutorial, Circuits Systems Signal Process. 42 (12) (2023) 7433–7466.
[34] Shaolei Guo, Yihao Wen, Xianqi Zhang, Haiyang Chen, Monthly runoff prediction using the VMD-LSTM-transformer hybrid model: a case study of the miyun reservoir in Beijing, J. Water Clim. Change 14 (9) (2023) 3221–3236.
[35] Ghadah Alkhayat, Rashid Mehmood, A review and taxonomy of wind and solar energy forecasting methods based on deep learning, Energy AI 4 (2021) 100060.
[36] Amir Mosavi, Mohsen Salimi, Sina Faizollahzadeh Ardabili, Timon Rabczuk, Shahaboddin Shamshirband, Annamaria R Varkonyi-Koczy, State of the art of machine learning models in energy systems, a systematic review, Energies 12 (7) (2019) 1301.
[37] Zheng Qian, Yan Pei, Hamidreza Zareipour, Niya Chen, A review and discussion of decomposition-based hybrid models for wind energy forecasting applications, Appl. Energy 235 (2019) 939–953.
[38] Changtong Wang, Zhaohua Liu, Hualiang Wei, Lei Chen, Hongqiang Zhang, Hybrid deep learning model for short-term wind speed forecasting based on time series decomposition and gated recurrent unit, Complex Syst. Model. Simul. 1 (4) (2021) 308–321.
[39] Zhong-kai Feng, Wen-jing Niu, Zheng-yang Tang, Zhi-qiang Jiang, Yang Xu, Yi Liu, Hai-rong Zhang, Monthly runoff time series prediction by variational mode decomposition and support vector machine based on quantum-behaved particle swarm optimization, J. Hydrol. 583 (2020) 124627.
[40] Wen-jing Niu, Zhong-kai Feng, Yu-bin Chen, Hai-rong Zhang, Chun-tian Cheng, Annual streamflow time series prediction using extreme learning machine based on gravitational search algorithm and variational mode decomposition, J. Hydrol. Eng. 25 (5) (2020) 04020008.
[41] Lingxiao Zhao, Zhiyang Li, Leilei Qu, Junsheng Zhang, Bin Teng, A hybrid VMD-LSTM/GRU model to predict non-stationary and irregular waves on the east coast of China, Ocean Eng. 276 (2023) 114136.
[42] Wenjin Sun, Shuyi Zhou, Jingsong Yang, Xiaoqian Gao, Jinlin Ji, Changming Dong, Artificial intelligence forecasting of marine heatwaves in the south China sea using a combined u-net and convlstm system, Remote Sens. 15 (16) (2023) 4068.
[43] Brandon J Bethel, Wenjin Sun, Changming Dong, Dongxia Wang, Forecasting hurricane-forced significant wave heights using a long short-term memory network in the caribbean sea, Ocean Sci. 18 (2) (2022) 419–436.
[44] Zhenbin Gao, Jie Zhang, The fluctuation correlation between investor sentiment and stock index using VMD-LSTM: Evidence from China stock market, North Am. J. Econ. Finance 66 (2023) 101915.
[45] Alexander J. McNeil, Modelling volatile time series with v-transforms and copulas, Risks 9 (1) (2021) 14.
[46] Ke Zhao, Dudu Guo, Miao Sun, Chenao Zhao, Hongbo Shuai, Short-term traffic flow prediction based on VMD and IDBO-LSTM, IEEE Access (2023).
[47] Etaf Alshawarbeh, Alanazi Talal Abdulrahman, Eslam Hussam, Statistical modeling of high frequency datasets using the ARIMA-ANN hybrid, Mathematics 11 (22) (2023) 4594.
[48] Chuan Li, Xinqiu Fang, Zhenguo Yan, Yuxin Huang, Minfu Liang, Research on gas concentration prediction based on the ARIMA-LSTM combination model, Processes 11 (1) (2023) 174.
[49] Dongyan Fan, Hai Sun, Jun Yao, Kai Zhang, Xia Yan, Zhixue Sun, Well production forecasting based on ARIMA-LSTM model considering manual operations, Energy 220 (2021) 119708.
[50] Ayaz Hussain Bukhari, Muhammad Asif Zahoor Raja, Muhammad Sulaiman, Saeed Islam, Muhammad Shoaib, Poom Kumam, Fractional neuro-sequential ARFIMA-LSTM for financial market forecasting, IEEE Access 8 (2020) 71326–71338.
[51] Athanasios Episcopos, Jefferson Davis, Predicting returns on Canadian exchange rates with artificial neural networks and EGARCH-M models, Neural Comput. Appl. 4 (1996) 168–174.
[52] Mauri Aparecido de Oliveira, The influence of ARIMA-GARCH parameters in feed forward neural networks prediction, Neural Comput. Appl. 20 (2011) 687–701.
[53] Werner Kristjanpoller, Marcel C. Minutolo, Forecasting volatility of oil price using an artificial neural network-GARCH model, Expert Syst. Appl. 65 (2016) 233–241.
[54] Monghwan Seo, Sungchul Lee, Geonwoo Kim, Forecasting the volatility of stock market index using the hybrid models with google domestic trends, Fluct. Noise Lett. 18 (01) (2019) 1950006.
[55] Monghwan Seo, Geonwoo Kim, Hybrid forecasting models based on the neural networks for the volatility of bitcoin, Appl. Sci. 10 (14) (2020) 4768.
[56] Bahareh Amirshahi, Salim Lahmiri, Hybrid deep learning and GARCH-family models for forecasting volatility of cryptocurrencies, Mach. Learn. Appl. 12 (2023) 100465.
[57] Yan Hu, Jian Ni, Liu Wen, A hybrid deep learning approach by integrating LSTM-ANN networks with GARCH model for copper price volatility prediction, Phys. A 557 (2020) 124907.
[58] Serkan Aras, Stacking hybrid GARCH models for forecasting bitcoin volatility, Expert Syst. Appl. 174 (2021) 114747.
[59] Werner Kristjanpoller, Marcel C. Minutolo, Gold price volatility: A forecasting approach using the Artificial Neural Network–GARCH model, Expert Syst. Appl. 42 (20) (2015) 7245–7251.
[60] Tim Bollerslev, Generalized autoregressive conditional heteroskedasticity, J. Econometrics 31 (3) (1986) 307–327.
[61] Guizhu Shen, Qingping Tan, Haoyu Zhang, Ping Zeng, Jianjun Xu, Deep learning with gated recurrent unit networks for financial sequence predictions, Procedia Comput. Sci. 131 (2018) 895–903.
[62] Rahul Dey, Fathi M. Salem, Gate-variants of gated recurrent unit (GRU) neural networks, in: 2017 IEEE 60th International Midwest Symposium on Circuits and Systems, MWSCAS, IEEE, 2017, pp. 1597–1600.
[63] John Kamwele Mutinda, Amos Kipkorir Langat, Capital asset pricing model: A renewed application on S&P 500 index, Asian J. Econ. Bus. Account. 24 (6) (2024) 226–239.
[64] Ailing Zeng, Muxi Chen, Lei Zhang, Qiang Xu, Are transformers effective for time series forecasting? in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 37, 2023, pp. 11121–11128.